

PYTHON AUTOMATION SCRIPT

OBJECTIVE

The objective of this project is to design and implement an intelligent file automation system using Python that simplifies routine file management tasks. The system is developed to automatically organize files, streamline file operations, and maintain detailed activity logs, reducing manual effort and improving efficiency. By automating repetitive processes, the project demonstrates how Python can be used to enhance productivity, ensure better file organization, and provide accurate tracking of performed operations.

DESCRIPTIVE

This project focuses on automating file management tasks using Python. The developed script helps organize files efficiently by performing predefined operations and maintaining a log of all activities. The automation process reduces manual effort, improves accuracy, and ensures that file-related tasks are completed in a structured manner. Additionally, the system records every operation in a log file, making it easy to track and monitor changes. The project demonstrates the practical application of Python for improving productivity through automation.

CODE SNIPPET

```
import os
import shutil
import logging

logging.basicConfig(
    filename="operations_log.txt",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)
```

```

FILE_TYPES = {
    "Images": [".jpg", ".jpeg", ".png", ".gif", ".bmp"],
    "Videos": [".mp4", ".mkv", ".avi", ".mov"],
    "Documents": [".pdf", ".docx", ".txt", ".xlsx", ".pptx"],
    "Music": [".mp3", ".wav", ".aac"],
    "Code": [".py", ".js", ".html", ".css", ".java"],
    "Others": []
}

```

```

def get_folder_name(extension):
    for folder, extensions in FILE_TYPES.items():
        if extension.lower() in extensions:
            return folder
    return "Others"

```

```

def sort_files(directory):
    print(f"\nSorting files in: {directory}")
    for filename in os.listdir(directory):
        file_path = os.path.join(directory, filename)
        if os.path.isdir(file_path):
            continue
        _, ext = os.path.splitext(filename)
        folder_name = get_folder_name(ext)
        target_folder = os.path.join(directory, folder_name)
        os.makedirs(target_folder, exist_ok=True)
        try:
            shutil.move(file_path, os.path.join(target_folder, filename))
            print(f"Moved: {filename} -> {folder_name}")
            logging.info(f"Moved: {filename} -> {folder_name}")
        except Exception as e:
            print(f"Error moving {filename}: {e}")

```

```
logging.error(f'Error moving {filename}: {e}')
```

```
def rename_files(directory):
```

```
    print(f'\nRenaming files in: {directory}')
```

```
    prefix = input("Enter prefix name (e.g. 'photo'): ")
```

```
    files = [f for f in os.listdir(directory) if os.path.isfile(os.path.join(directory, f))]
```

```
    for i, filename in enumerate(files):
```

```
        ext = os.path.splitext(filename)[1]
```

```
        new_name = f'{prefix}_{i+1}{ext}'
```

```
        try:
```

```
            os.rename(
```

```
                os.path.join(directory, filename),
```

```
                os.path.join(directory, new_name)
```

```
            )
```

```
            print(f'Renamed: {filename} -> {new_name}')
```

```
            logging.info(f'Renamed: {filename} -> {new_name}')
```

```
        except Exception as e:
```

```
            print(f'Error renaming {filename}: {e}')
```

```
def delete_empty_folders(directory):
```

```
    print(f'\nCleaning empty folders in: {directory}')
```

```
    for folder in os.listdir(directory):
```

```
        folder_path = os.path.join(directory, folder)
```

```
        if os.path.isdir(folder_path) and not os.listdir(folder_path):
```

```
            try:
```

```
                os.rmdir(folder_path)
```

```
                print(f'Deleted empty folder: {folder}')
```

```
                logging.info(f'Deleted empty folder: {folder}')
```

```
            except Exception as e:
```

```
                print(f'Error deleting {folder}: {e}')
```

```
def main():
```

```
print("=== File Automation Tool ===")
directory = input("\nEnter the folder path to work on: ")
if not os.path.exists(directory):
    print("That folder does not exist!")
    return
print("\nWhat do you want to do?")
print("1. Sort files by type")
print("2. Rename files")
print("3. Delete empty folders")
print("4. Do all")
choice = input("\nEnter choice (1/2/3/4): ")
if choice == "1":
    sort_files(directory)
elif choice == "2":
    rename_files(directory)
elif choice == "3":
    delete_empty_folders(directory)
elif choice == "4":
    sort_files(directory)
    rename_files(directory)
    delete_empty_folders(directory)
else:
    print("Invalid choice!")
print("\nDone! Check operations_log.txt for details.")

if __name__ == "__main__":
    main()
```

SCREENSHOTS

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kanja>mkdir C:\file_automation
C:\Users\kanja>cd C:\file_automation
C:\file_automation>notepad file_organizer.py
C:\file_automation>cd C:\file_automation
C:\file_automation>notepad file_organizer.py
C:\file_automation>python file_organizer.py
=== File Automation Tool ===

Enter the folder path to work on: C:\file_automation

What do you want to do?
1. Sort files by type
2. Rename files
3. Delete empty folders
4. Do all

Enter choice (1/2/3/4): 1

Sorting files in: C:\file_automation
Moved: file_organizer.py -> Code
Error moving operations_log.txt: [WinError 32] The process cannot access the file because it is being used by another process: 'C:\\file_automation\\operations_log.txt'

Done! Check operations_log.txt for details.

C:\file_automation>cd Code
C:\file_automation\Code>python file_organizer.py
=== File Automation Tool ===

Enter the folder path to work on: C:\file_automation

What do you want to do?
1. Sort files by type
2. Rename files
3. Delete empty folders
4. Do all
```

```
Command Prompt

Enter choice (1/2/3/4): 2

Renaming files in: C:\file_automation
Enter prefix name (e.g. 'photo'): anjali
Renamed: operations_log.txt -> anjali_1.txt

Done! Check operations_log.txt for details.

C:\file_automation\Code>python file_organizer.py
=== File Automation Tool ===

Enter the folder path to work on: C:\file_automation

What do you want to do?
1. Sort files by type
2. Rename files
3. Delete empty folders
4. Do all

Enter choice (1/2/3/4): 3

Cleaning empty folders in: C:\file_automation

Done! Check operations_log.txt for details.

C:\file_automation\Code>type C:\file_automation\anjali_1.txt
2026-06-02 19:41:46,653 ~ Moved: file_organizer.py -> Code
2026-06-02 19:41:46,671 ~ Error moving operations_log.txt: [WinError 32] The process cannot access the file because it is being used by another process: 'C:\\file_automation\\operations_log.txt'

C:\file_automation\Code>git --version
git version 2.53.0.windows.2

C:\file_automation\Code>cd C:\file_automation\Code
C:\file_automation\Code>git init
Initialized empty Git repository in C:/file_automation/Code/.git/
C:\file_automation\Code>git add .
```

```
Command Prompt
C:\file_automation\Code>git init
Initialized empty Git repository in C:/file_automation/Code/.git/

C:\file_automation\Code>git add .

C:\file_automation\Code>git commit -m "first commit"
[master (root-commit) a2c4940] first commit
2 files changed, 101 insertions(+)
create mode 100644 file_organizer.py
create mode 100644 operations_log.txt

C:\file_automation\Code>https://github.com/anjali-kadupukutti/file-automation.git
'https:' is not recognized as an internal or external command,
operable program or batch file.

C:\file_automation\Code>git remote add origin https://github.com/anjali-kadupukutti/file-automation.git

C:\file_automation\Code>git remote -v
origin https://github.com/anjali-kadupukutti/file-automation.git (fetch)
origin https://github.com/anjali-kadupukutti/file-automation.git (push)

C:\file_automation\Code>git add .

C:\file_automation\Code>git commit -m "Initial commit"
On branch master
nothing to commit, working tree clean

C:\file_automation\Code>git push -u origin master
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/anjali-kadupukutti/file-automation.git'

C:\file_automation\Code>git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.40 KiB | 477.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/anjali-kadupukutti/file-automation.git
 * [new branch] master -> master
branch 'master' set up to track 'origin/master'.

C:\file_automation\Code>
```

```
operations_log.txt
File Edit View
H1 H2 H3 B I S G A
2026-06-02 19:43:32,468 - Renamed: operations_log.txt -> anjali_1.txt
```

anjali-kadupukutti / file-automation

file-automation Public

master 1 Branch 0 Tags

anjali-kadupukutti Create README.md 3934x50 · 24 minutes ago 2 Commits

README.md	Create README.md	24 minutes ago
file_organizer.py	first commit	34 minutes ago
operations_log.txt	first commit	34 minutes ago

File Automation

Python script that automatically organizes files and logs operations.

Features

- Renames files
- Organizes files
- Generates operation logs

Files

About

A Python automation script that sorts, renames, and cleans files. Uses OS module, exception handling, and generates logs for all operations.

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors

No contributors

GITHUB REPOSITORY LINK

<https://github.com/anjali-kadupukutti/file-automation>

CONCLUSION

This project successfully demonstrates the use of Python for automating file management tasks. The developed system reduces manual effort by performing file operations efficiently and maintaining detailed logs of all activities. Through this project, practical experience was gained in Python programming, file handling, automation techniques, and version control using Git and GitHub. The solution improves productivity and showcases the effectiveness of automation in managing repetitive tasks.